CSCB07: Software Design
Lecture 1

Rafael F. Toledo

Fall 2026

1 Introduction to C++

Let's compare a program that prints `"Hello World"` written in C and C++.

Hello world in C:

```c
#include <stdio.h>
int main() {
printf("Hello␣world\n");
return 0;
}
```

Hello world in C++:

```cpp
import <iostream>;
//Testing comments
using namespace std;
int main() {
    cout << "Hello␣world" << endl;
    return 0;
}
```

In C++,

- main
  - must return int;
  - `void main() { . . . }` not legal in C++.
- return stmt
  - returns status code to OS (`$?`)
  - can be omitted from main (0 assumed)
- `stdio`, `printf` are still available in C++
- preferred C++ I/O: header <`iostream`>:

```cpp
std::cout << . . . << . . . << . . .
```

- `std::endl` = end of line + flush output buffer
- `using namespace std;` lets you omit the `std::` prefix

Most C programs work in C++ (or require minimal changes)

1.1 Constants

`const` variables cannot be modified later in the program.

```
const int maxGrade = 100; // must be initialized
```

Advice: Declare as many things const as you can. Helps catch errors.

1.2 Data Structs

Data structs are user-defined data types or a set of data elements (or members) grouped together under one name.

```
struct Node {
    int data;
    /* struct */ Node *next; //no need for the keyword struct
}; // don't forget the semicolon!
...
Node n{5, nullptr};
const Node n2 = n; // immutable copy of n
```

- `nullptr` is syntax for null pointers in C++.
- Do NOT say `NULL` or 0 in this course!

1.3 Parameter Passing

```
void inc(int n) { ++n; }
// . . .
int x = 5;
inc(x);
cout << x << endl; // 5
```

In this example, the value of `x` remains unchanged. That happens because `inc` gets a copy of `x` and mutates the copy, not the original variable.

If a function needs to mutate an argument, we can pass the argument as a pointer:

```
void inc (int *n) { ++*n; }
// . . .
int x = 5;
inc(&x);
cout << x << endl; // 6
```

The address of `x` is passed by value and `inc` changes the data at that address. In C++, there is a new type that can also be used for parameter passing: references.

1.4 References

```
int y = 10;
int &z = y; // z is an lvalue reference to an int variable
```

A reference is like a const pointer, similar to `int * const z = &y;` So we cannot change which variable the reference points to after its declaration. References also have automatic dereferencing, so there is no need to use `*` to access the value being referenced. For example, if we want to reassign the value stored in `y`, we can run:

```
z = 12; // (NOT *z = 12;) now y is equal to 12
```

In all cases, `z` behaves like `y`, meaning that `z` works as an alias (another name) for `y`. Thus, taking the address of `z`, like `int *p = &z;` returns the address of `y`.

Note that we have learned a new meaning to the ampersand symbol (&). It is important to highlight the difference between them:

- Reference: whenever & occurs as a part of a type (e.g., `int &z = ...`)
- Address-of: whenever & occurs in an expression (e.g., `... = &z`)

Things you cannot do with *lvalue references*:

- Leave them uninitialized, e.g., `int &x;`
  - must be initialized with something that has an address (an lvalue), since references are pointers.

```
int &x = 3; // NO
int &x = y + z; // NO
int &x = y; // OK
```

- Create a pointer to a reference, e.g., `int &*x;`
  - A reference to a pointer is OK, e.g., `int *&x;`
- Create a reference to a reference, e.g., `int &&r` (this actually means something different - later).
- Create an array of references, e.g., `int &r[3] = {n, n, n};`

What can you do with references? Pass them as function parameters:

```
void inc (int &n) { // Constant ptr n to an argument x.
                    // Changes to n affect x.
    ++n; // No pointer dereference
}
. . .
int x = 5;
inc(x);
cout << x << endl; // 6
```

Now we have more options for parameter passing, and the choice can affect the cost and constraints of executing our code. Consider, for example, a function that takes a very large data structure as a parameter. If we pass this data structure by value, we may make our code very slow because it has to make a copy of the large data structure. If we pass it by reference, the code runs faster because it avoids copying, but we give the function permission to change the parameter value to whatever it wants. Depending on the situation, we may want to restrict such access while still avoiding the copy. To perform that, we can pass the argument by *const reference*:

```
struct ReallyBig { ... };
int f (ReallyBig rb) { ... } // Copy. Potentially slow.
int g (ReallyBig &rb) { ... } // Alias. Fast.
                              // But could change rb in the
                                 function.
int h (const ReallyBig &rb) { ... } // Fast. No copy.
                                    // Parameter cannot be
                                       changed.
```

If you are passing a parameter by const reference to a function that must make local changes to a copy of the parameter. Then, the function must make a copy of it inside its body:

```
int h(const ReallyBig &rb) {
ReallyBig rb2 = rb;
    // mutate rb2
    // . . .
}
```

However, if you have to make a copy anyway, it's better to just use *pass-by-value* and have the compiler make it for you (maybe it can optimize something).

Advice: Prefer *pass-by-const-ref* over *pass-by-value* for anything larger than a pointer, unless the function needs to make a copy anyway. In that case, use *pass-by-value*.

Extra fun fact about references:

```
int f (int &n);
int g (const int &n);

f(5); // NO. Can't initialize lvalue ref n to a literal value
      // If n changes, can't change the literal 5

g(5); // OK. Since n can never be changed, the compiler allows
   this
```